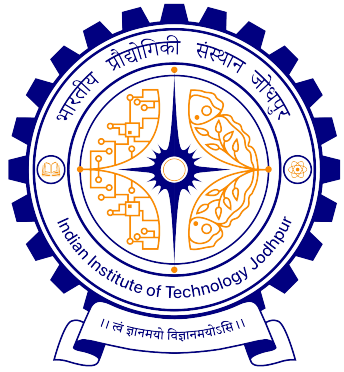




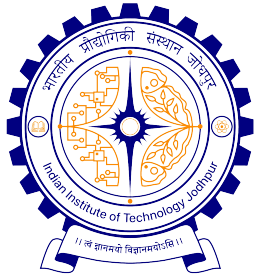
Introduction to Computer Science (CSL1010)

C Basics– Sorting Techniques L16

Suchetana Chakraborty

Department of Computer Science, IIT Jodhpur

Spring, 2026



Sorting Task

- **Given an array**

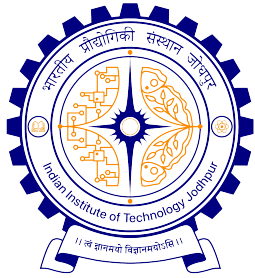
$x[0], x[1], \dots, x[\text{size}-1]$

reorder entries so that

$x[0] \leq x[1] \leq \dots \leq x[\text{size}-1]$

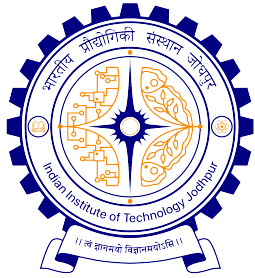
Here, List is in non-decreasing order.

- **We can also sort a list of elements in non-increasing order.**



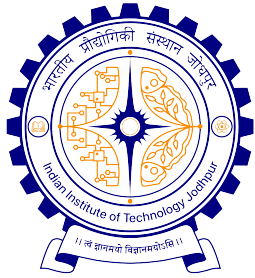
Example

- Original list:
 - 10, 30, 20, 80, 70, 10, 60, 40, 70
- Sorted in non-decreasing order:
 - 10, 10, 20, 30, 40, 60, 70, 70, 80
- Sorted in non-increasing order:
 - 80, 70, 70, 60, 40, 30, 20, 10, 10



Factors Influencing Sorting Technique

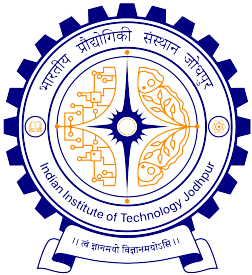
- How to rearrange a given set of data?
- Which data structures are more suitable to store data prior to their sorting?
- How fast the sorting can be achieved?
- How sorting can be done in a memory constraint situation?
- How to sort various types of data?



Sorting by Comparison

Basic operation involved in this type of sorting technique is comparison. A data item is **compared with other items** in the list of items in order **to find its place** in the sorted list.

- Insertion
- **Selection*** (already seen in previous classes)
- Exchange
- Enumeration



Sorting by Comparison

Sorting by comparison – Insertion:

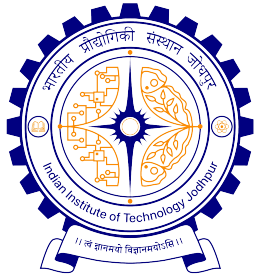
- From a given list of items, one item is considered at a time. The item chosen is then inserted into an appropriate position relative to the previously sorted items. The item can be inserted into the same list or to a different list.

e.g.: Insertion sort

Sorting by comparison – Selection:

- First the smallest (or largest) item is located and it is separated from the rest; then the next smallest (or next largest) is selected and so on until all items are separated.

e.g.: Selection sort, Heap sort



Sorting by Comparison

Sorting by comparison – Exchange:

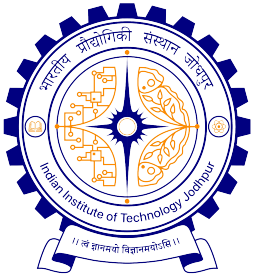
- If two items are found to be out of order, they are interchanged. The process is repeated until no more exchange is required.

e.g.: Bubble sort, Shell Sort, Quick Sort

Sorting by comparison – Enumeration:

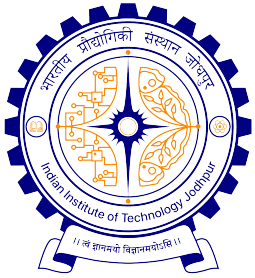
- Two or more input lists are **merged** into an output list and while merging the items, an input list is chosen following the required sorting order.

e.g.: Merge sort

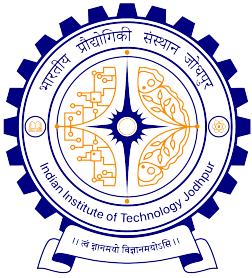


Sorting by Distribution

- No key comparison takes place
- All items under sorting are distributed over an auxiliary storage space based on the constituent element in each and then grouped them together to get the sorted list.
- Distributions of items based on the following choices
 - ✓ **Radix** - An item is placed in a space decided by the bases (or radix) of its components with which it is composed of.
 - ✓ **Counting** - Items are sorted based on their relative counts.
 - ✓ **Hashing** - Items are hashed, that is, dispersed into a list based on a hash function.

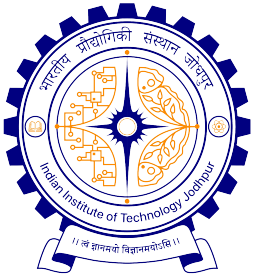


Insertion Sort

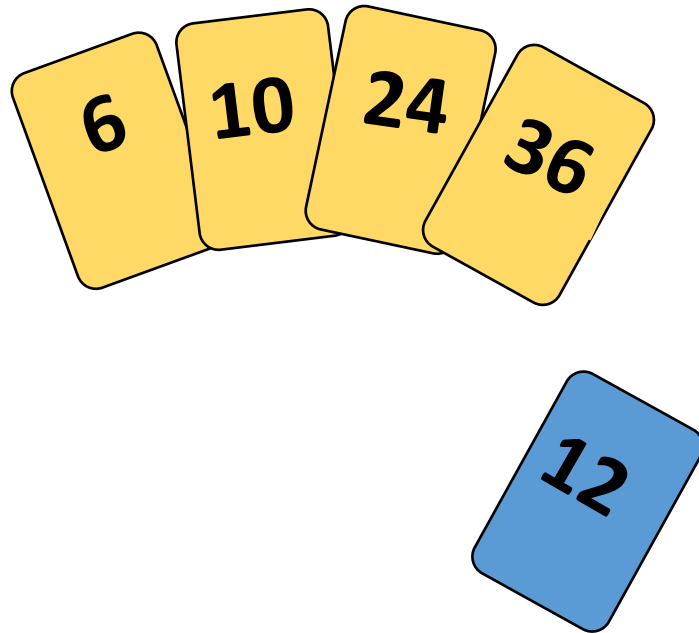


Insertion Sort

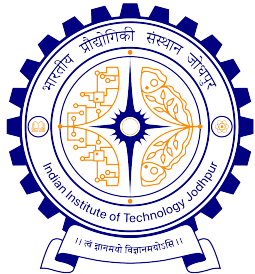
- Idea: like sorting a hand of playing cards
 - Start with an empty left hand and the cards facing down on the table.
 - Remove one card at a time from the table, and insert it into the correct position in the left hand
 - compare it with each of the cards already in the hand, from right to left
 - The cards held in the left hand are sorted
 - these cards were originally the top cards of the pile on the table



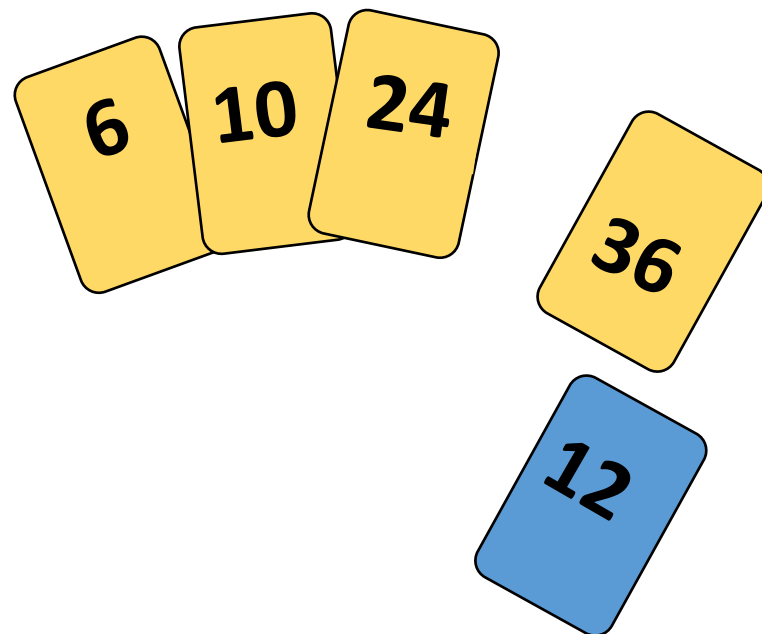
Insertion Sort



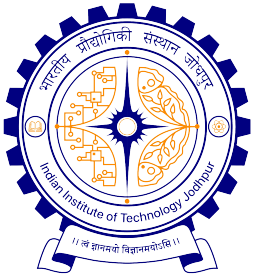
To insert 12, we need to make room for it by moving first 36 and then 24.



Insertion Sort

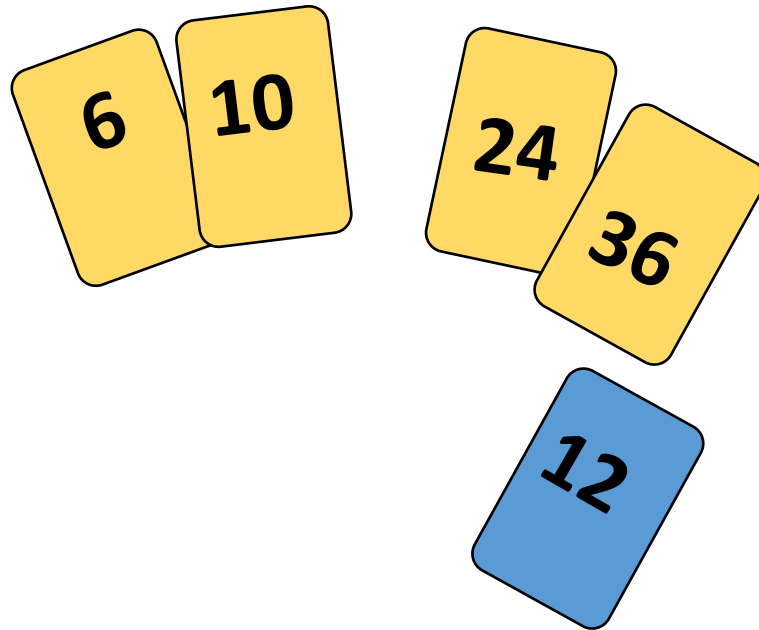


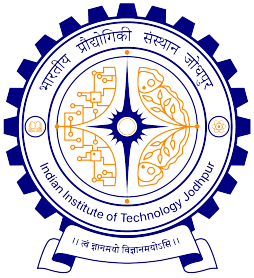
To insert 12, we need to make room for it by moving first 36 and then 24.



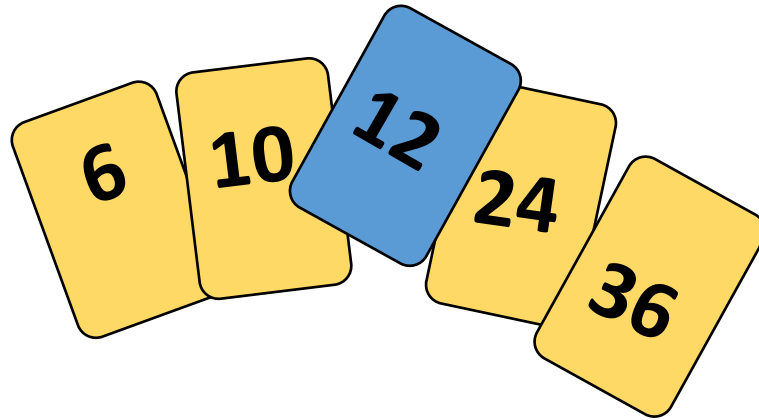
Insertion Sort

To insert 12, we need to make room for it by moving first 36 and then 24.

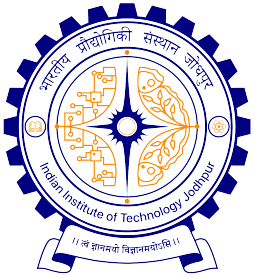




Insertion Sort



To insert 12, we need to make room for it by moving first 36 and then 24.



Insertion Sort

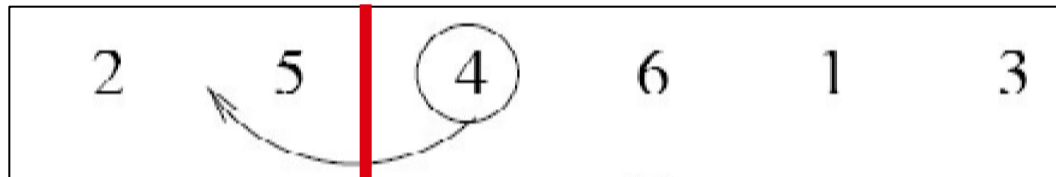
input array

5 2 4 6 1 3

at each iteration, the array is divided in two sub-arrays:

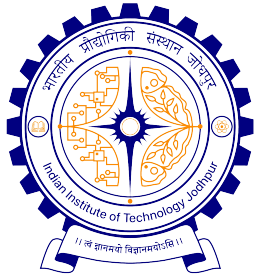
left sub-array

right sub-array

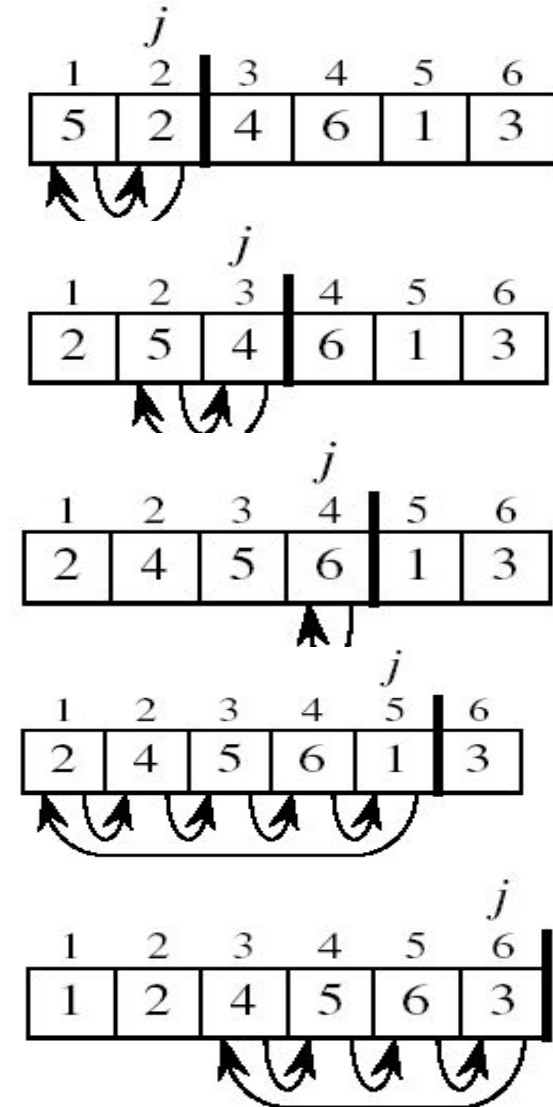
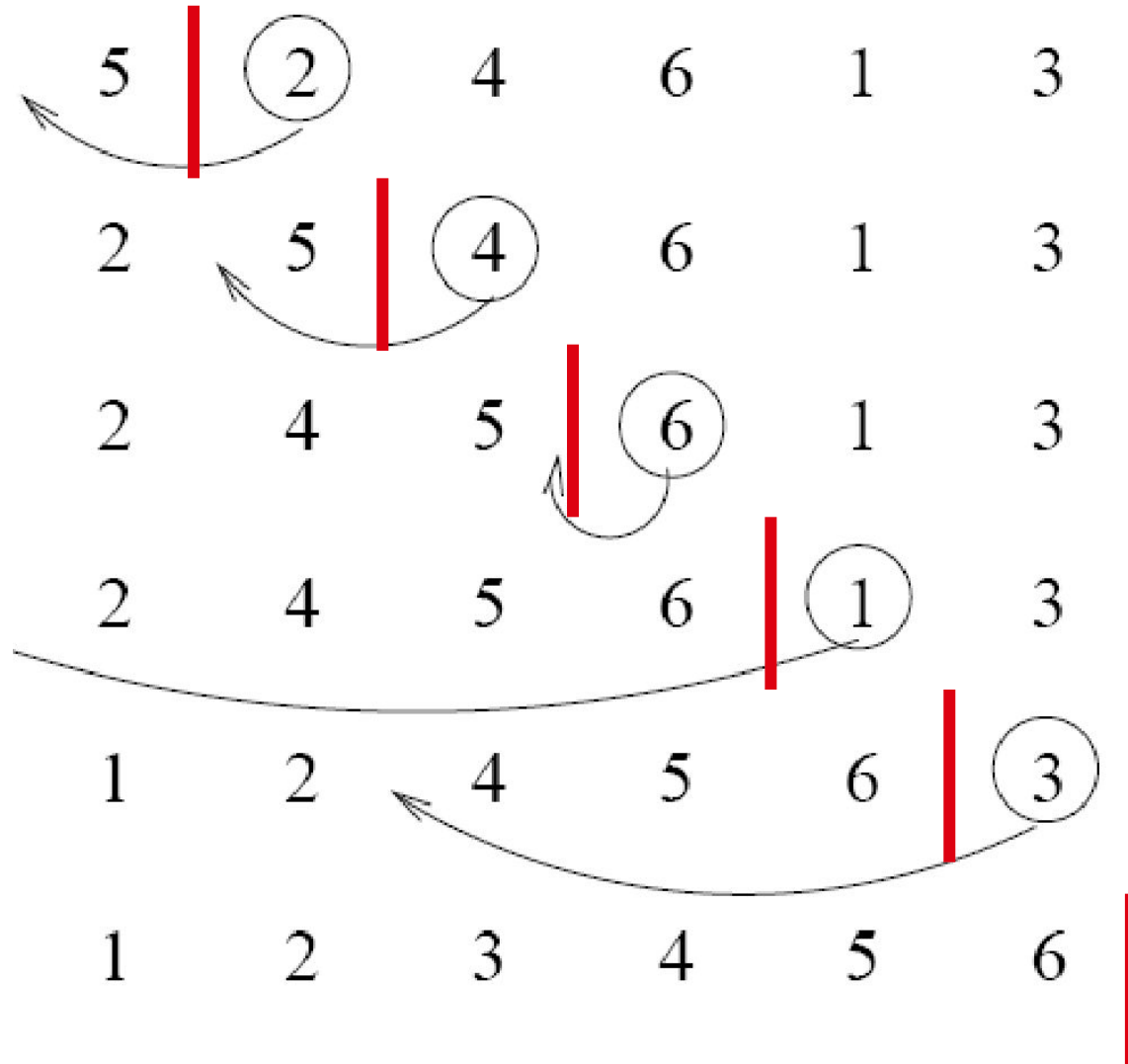


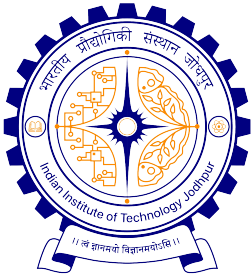
sorted

unsorted



Insertion Sort





Insertion Sort

```
void insertionSort (int list[], int size) {  
    int i,j,item;  
    for (i=1; i<size; i++){  
  
        item = list[i] ;  
        /* Move elements of list[0..i-1], that are greater  
        than item, to one position ahead of their current  
        position */  
        for (j=i-1; (j>=0)&& (list[j] > item); j--)  
            list[j+1] = list[j];  
        list[j+1] = item ;  
    }  
}
```



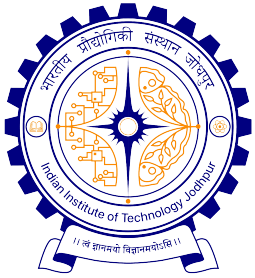
Insertion Sort

```
int main() {  
    int x[ ]={-45,89,-65,87,0,3,-23,19,56,21,76,-50};  
  
    int i;  
    for(i=0;i<12;i++)  
        printf("%d ",x[i]);  
    printf("\n");  
  
    insertionSort(x,12);  
  
    for(i=0;i<12;i++)  
        printf("%d ",x[i]);  
    printf("\n");  
}
```

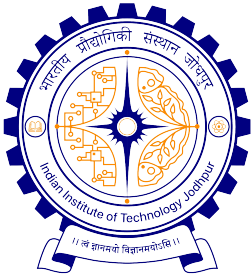
OUTPUT

-45 89 -65 87 0 3 -23 19 56 21 76 -50

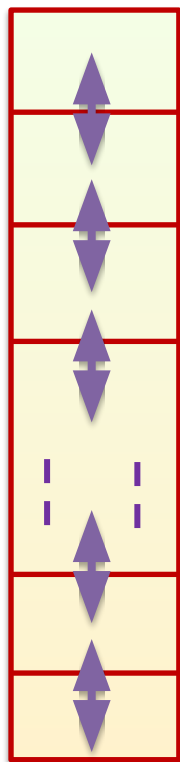
-65 -50 -45 -23 0 3 19 21 56 76 87 89



Bubble Sort



Bubble Sort

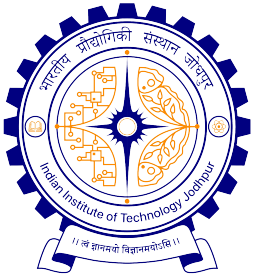


In every iteration
heaviest element drops
at the bottom.

The bottom
moves upward.

The sorting process proceeds in several passes.

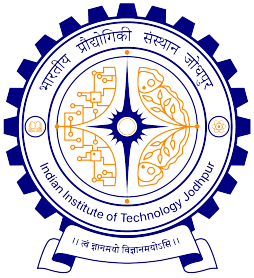
- In every pass we go on **comparing neighbouring pairs**, and **swap** them if out of order.
- In every pass, the largest of the elements under considering will **bubble** to the top (i.e., the right).



Bubble Sort

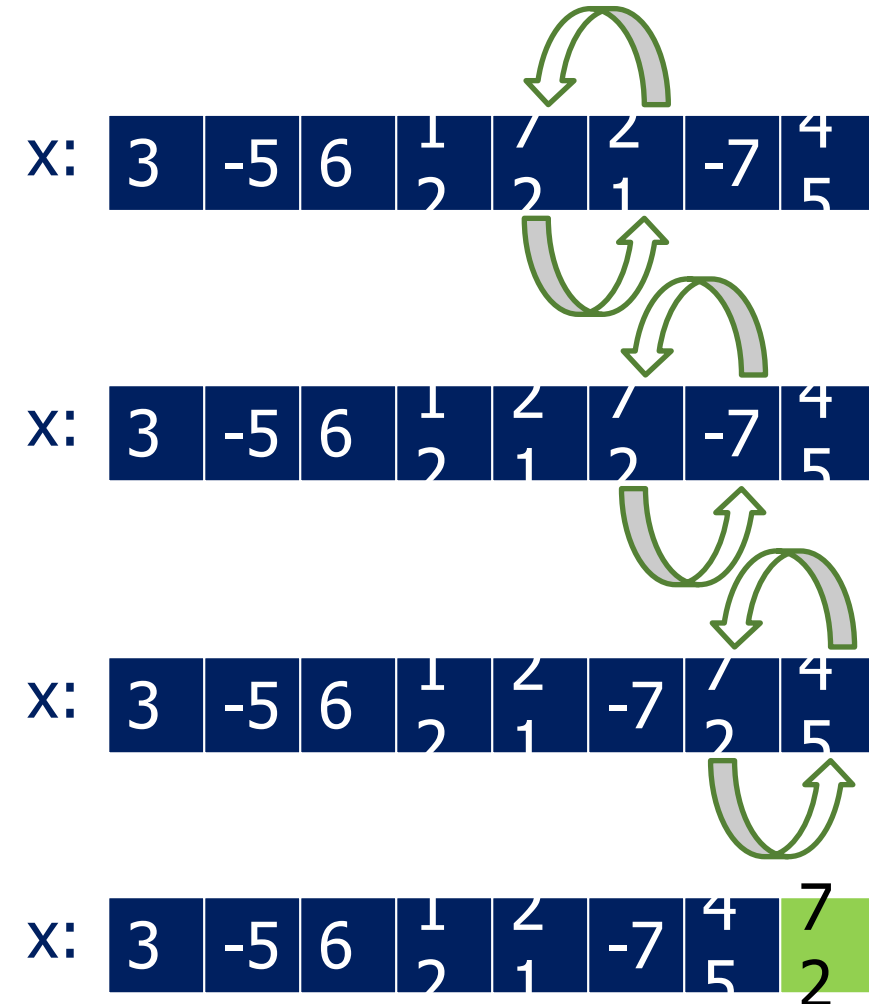
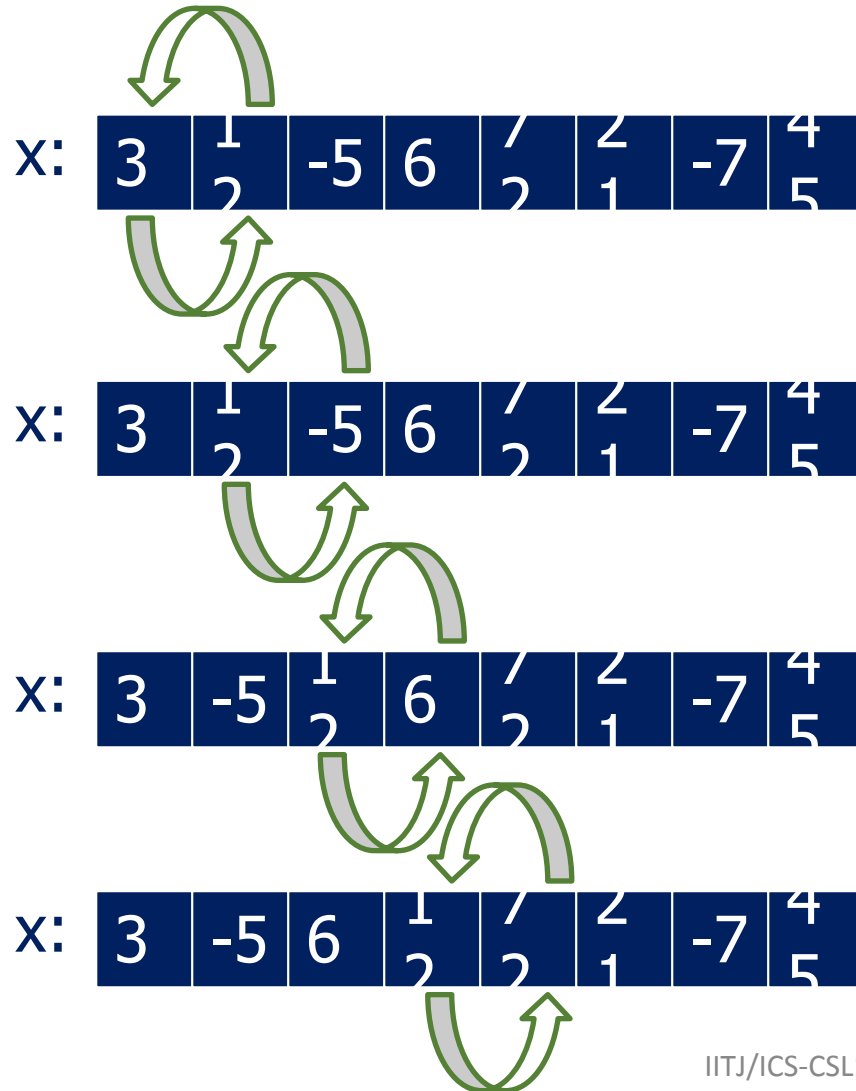
- How the passes proceed?

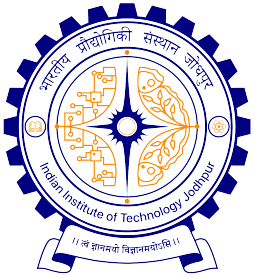
- In **pass 1**, we consider index **0** to **n-1**.
- In **pass 2**, we consider index **0** to **n-2**.
- In **pass 3**, we consider index **0** to **n-3**.
-
-
- In **pass n-1**, we consider index **0** to **1**.



Bubble Sort

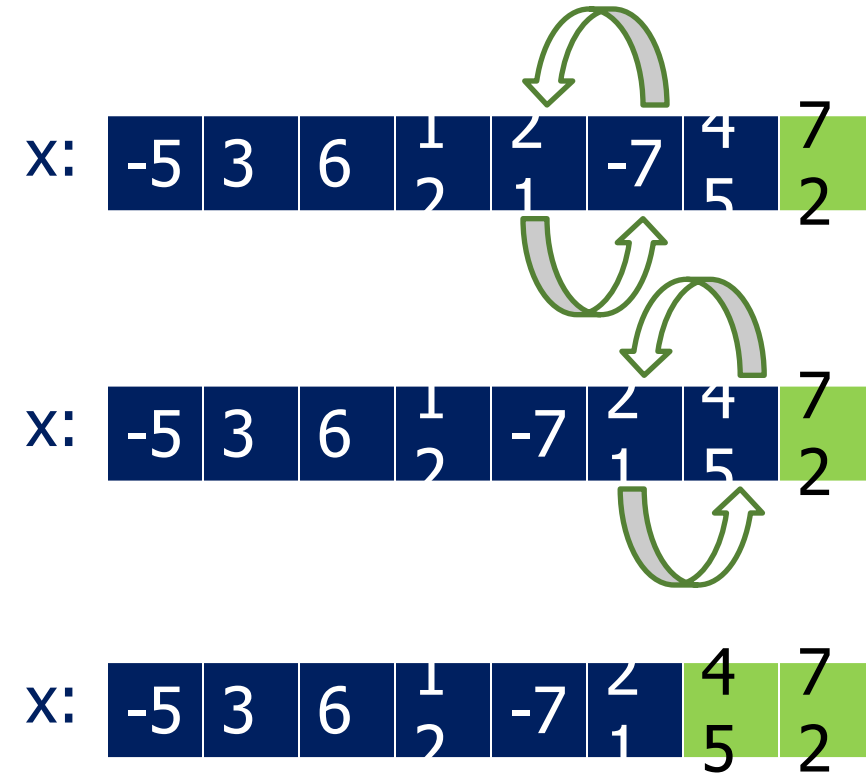
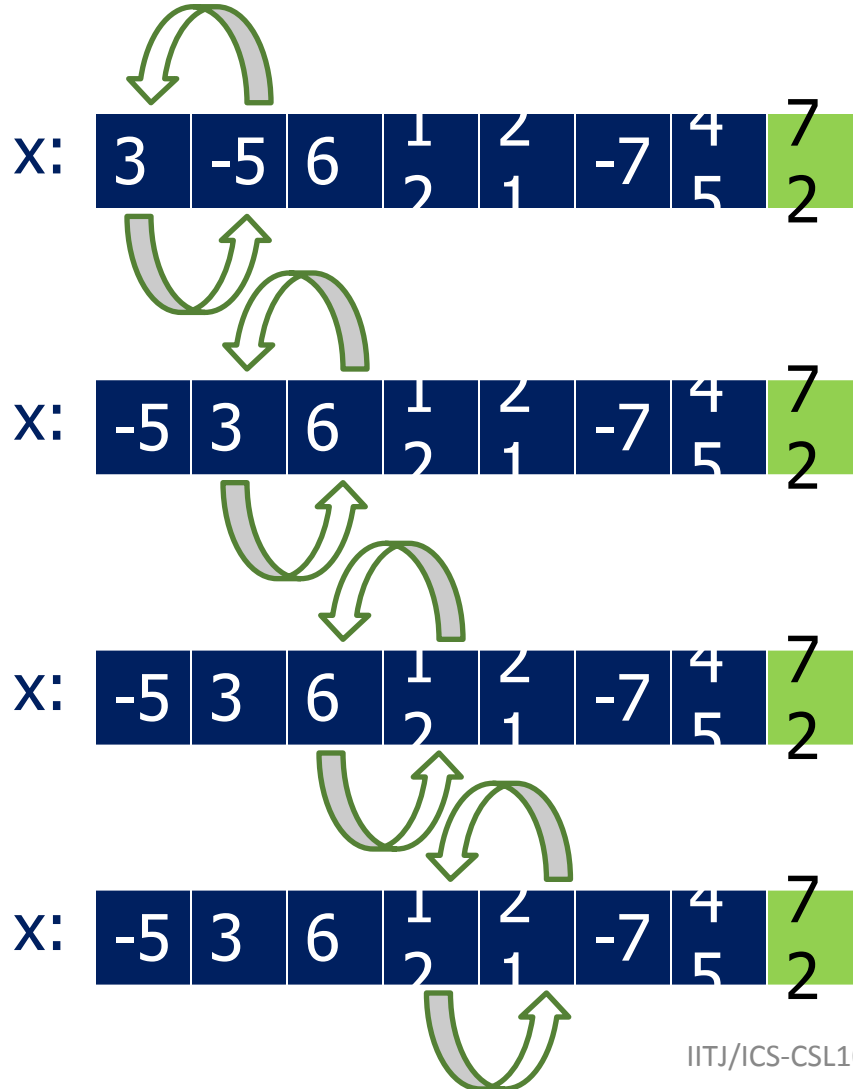
Pass: 1

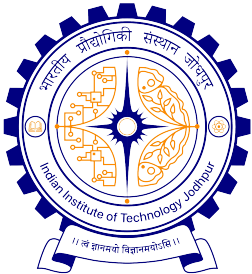




Bubble Sort

Pass: 2

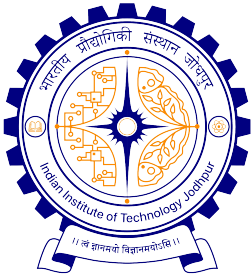




Bubble Sort

```
void swap(int *x, int *y){
    int tmp = *x;
    *x = *y;
    *y = tmp;
}

void bubble_sort(int x[], int n){
    int i,j;
    for (i=n-1; i>0; i--)
        for (j=0; j<i; j++)
            if (x[j] > x[j+1])
                swap(&x[j], &x[j+1]);
}
```

Bubble Sort

```
int main(){
    int x[ ]={-45,89,-65,87,0,3,-23,19,56,21,76,-50};
    int i;
    for(i=0;i<12;i++)
        printf("%d ",x[i]);
    printf("\n");

    bubble_sort(x,12);

    for(i=0;i<12;i++)
        printf("%d ",x[i]);
    printf("\n");
}
```

OUTPUT

-45 89 -65 87 0 3 -23 19 56 21 76 -50
-65 -50 -45 -23 0 3 19 21 56 76 87 89